

Εντολές Διακλάδωσης

(και ο μηχανισμός κλήσης συναρτήσεων)

<https://mixstef.github.io/courses/comparch/>

Μ.Στεφανιδάκης



Χρήση Διευθύνσεων Μνήμης

- Προσπέλαση δεδομένων (μεταβλητές προγραμμάτων)
 - Εντολές μηχανής που διαβάζουν ή γράφουν δεδομένα σε διευθύνσεις μνήμης
 - Διάφοροι τρόποι σχηματισμού διεύθυνσης μνήμης δεδομένων για την υποβοήθηση των γλωσσών προγραμματισμού
- Ανάκληση εντολών (ροή εκτέλεσης)
 - Αυτόματη ενημέρωση του καταχωρητή **program counter (PC)** στη διεύθυνση μνήμης της επόμενης προς εκτέλεση εντολής
 - Οι **εντολές διακλάδωσης (branches/jumps)** αλλάζουν τιμή στον **PC**: φορτώνουν στον PC μια νέα διεύθυνση μνήμης
 - Όπου θα μεταφερθεί η ροή εκτέλεσης

Τι περιγράφει μια εντολή διακλάδωσης

- Πότε θα «εκτελεστεί» η εντολή διακλάδωσης
 - Μια **συνθήκη** που πρέπει να ισχύει για να φορτωθεί η νέα διεύθυνση στον PC
 - Αν δεν ισχύει η συνθήκη, ο PC συνεχίζει με τη διεύθυνση της εντολής που ακολουθεί την εντολή διακλάδωσης
 - Η συνθήκη αυτή υπολογίζεται:
 - Είτε στην **ίδια την εντολή διακλάδωσης**
“if a == b then goto X”
 - Είτε σε **προηγούμενη εντολή**
“compare a and b;
if equal then goto X”
 - Υπάρχουν και εντολές διακλάδωσης **χωρίς συνθήκη**
“goto X”

Τι περιγράφει μια εντολή διακλάδωσης (2)

- Πού θα μεταφερθεί η ροή του προγράμματος εάν «εκτελεστεί» η εντολή διακλάδωσης
 - Ποια είναι η νέα διεύθυνση που θα φορτωθεί στον PC
 - Η νέα διεύθυνση μπορεί να περιγράφεται:
 - Είτε ως απόλυτη διεύθυνση
“πήγαινε στη διεύθυνση 1000”
 - Είτε ως σχετική διεύθυνση ως προς την τρέχουσα θέση, δηλ. την τρέχουσα τιμή του PC
“πήγαινε 40 bytes πριν την τρέχουσα εντολή”
 - Όταν οι διακλαδώσεις (αλλά και τα στατικά/global δεδομένα) χρησιμοποιούν μόνο σχετικές (σε σχέση με τον PC) διευθύνσεις, το πρόγραμμα μπορεί να φορτωθεί σε οποιαδήποτε θέση μνήμης χωρίς αλλαγές (position independent code)

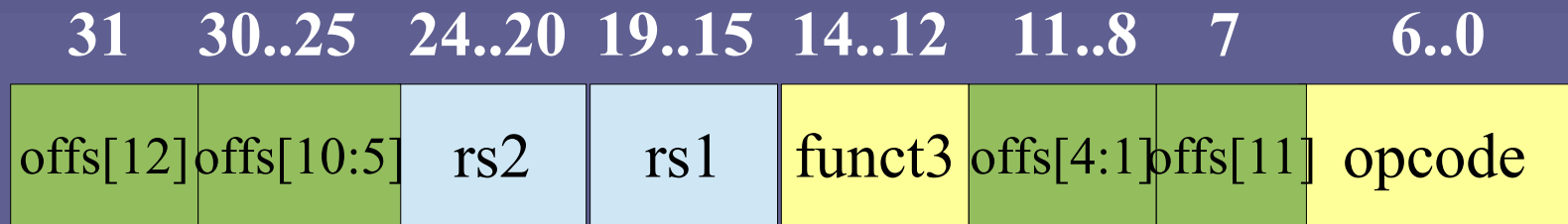
Κλήσεις και επιστροφές από συναρτήσεις

- Είναι και αυτές είδος διακλάδωσης
 - Ο PC παίρνει ως νέα τιμή το **σημείο εισόδου στη συνάρτηση** ή επιστρέφει αμέσως **μετά το σημείο κλήσης**
- Ποια η διαφορά από τις «κανονικές» εντολές διακλάδωσης;
 - Δυνατότητα αποθήκευσης του **αρχικού PC** (πριν τη διακλάδωση)
 - Σε καταχωρητή ή σε θέση μνήμης (runtime stack)
 - Για να χρησιμοποιηθεί στο τέλος της συνάρτησης ως **διεύθυνση επιστροφής** (return address)

Παράδειγμα: RISC-V (32 και 64-bit)

- Εντολές διακλάδωσης υπό συνθήκη
 - Ακολουθείται η τακτική «compare-and-branch»: υπολογισμός συνθήκης και διακλάδωση εκτελούνται **στην ίδια εντολή**
 - Υπολογισμός συνθήκης = σύγκριση τιμών 2 καταχωρητών
 - Επιλογή σύγκρισης αριθμών **με** και **χωρίς πρόσημο**
 - Εάν ισχύει η ζητούμενη συνθήκη → προστίθεται στον PC μια ακέραια σταθερά (offset με πρόσημο) των **13 bits**
 - Η μετατόπιση μπορεί να είναι έως **$\pm 4\text{KB}$**
- Εντολές διακλάδωσης χωρίς συνθήκη
 - $\text{PC} \leftarrow \text{PC} + \text{ακέραια σταθερά των 21 bits (μετατόπιση } \pm 1\text{MB)}$
 - $\text{PC} \leftarrow \text{ακέραια σταθερά των 12 bits} + \text{το περιεχόμενο ενός καταχωρητή (μετατόπιση σε όλο το εύρος διευθύνσεων)}$

RISC-V Conditional Branches (B-type)



funct3	σύγκριση (cmp)
000	BEQ (==)
001	BNE (!=)
100	BLT (<)
101	BGE (>=)
110	BLTU (<)
111	BGEU (>=)

είδος σύγκρισης

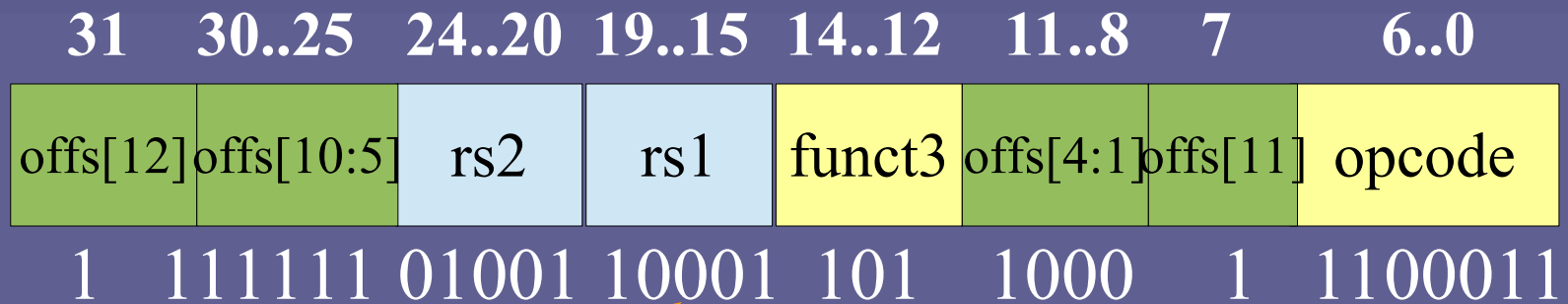
1100011

if $rs1 \text{ } cmp \text{ } rs2$ is true:

$pc \leftarrow pc \pm offset$

Οι εντολές RISC-V είναι 32 bits και συμπιεσμένες 16 bits → κάθε εντολή βρίσκεται πάντα σε ζυγή διεύθυνση → το $offs[0]$ είναι πάντα 0 και δεν αποθηκεύεται στην εντολή

Παράδειγμα



bge x17, x9, -16

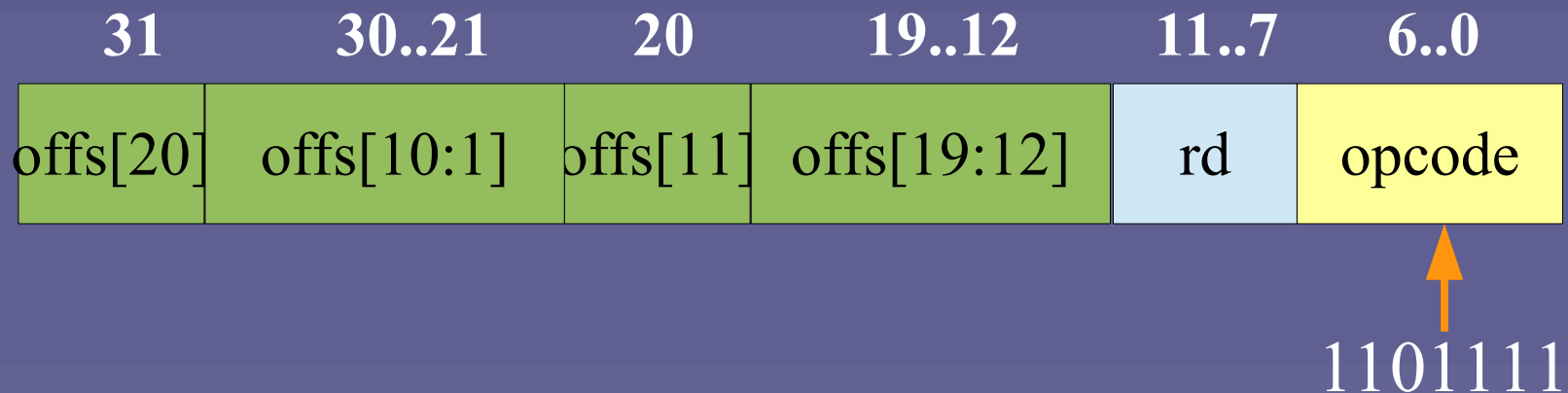
// if x17 ≥ x9 then pc = pc-16

bytes εντολής: FE 98 D8 E3

Σημ.: -16 (σε 13 bits) = 1 1111 1111 000(0)

Δεν ξεχνάμε και το bit0 που θεωρείται πάντα 0

RISC-V Jump and link (JAL, J-type)

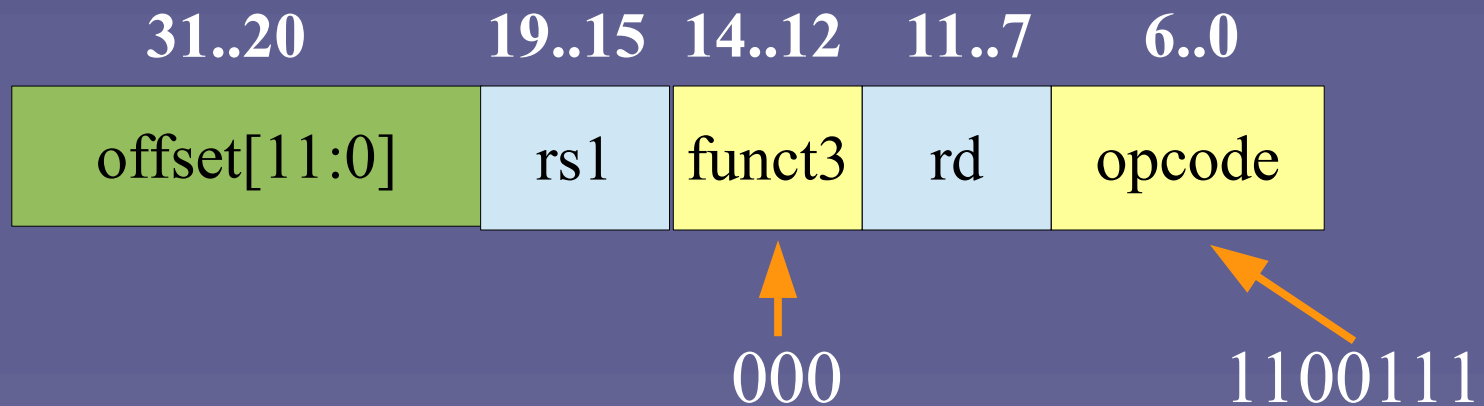


$rd \leftarrow pc + 4 \text{ (next instruction)}$
 $pc \leftarrow pc \pm \text{offset}$

Όπως και πριν, το offs[0] είναι πάντα 0 και δεν αποθηκεύεται

Για διακλάδωση χωρίς συνθήκη σε σχετική προς τον PC διεύθυνση

RISC-V Jump and link register (JALR, I-type)



$rd \leftarrow pc + 4$ (next instruction)

$pc \leftarrow rs1 \pm offset$

Το offset προστίθεται στον rs1 και στην τιμή που προκύπτει το bit0 γίνεται 0

Για διακλάδωση χωρίς συνθήκη, η χρήση εξαρτάται από την αρχικοποίηση του rs1

Μεγάλες σταθερές (U-type)



0110111 (lui)

0010111 (auipc)

lui rd, imm // $rd \leftarrow imm \ll 12$

auipc rd, imm // $rd \leftarrow pc + (imm \ll 12)$

Εντολές για να βάλουμε σταθερές των 20 bits στο «πάνω» μέρος ενός καταχωρητή. Συνδυάζονται με άλλες εντολές που φορτώνουν τα 12 «χαμηλότερα» bits του καταχωρητή (π.χ. ADDI, αλλά προσοχή στην επέκταση προσήμου!)

Χρήση lui + jalr → διακλάδωση σε απόλυτη διεύθυνση

Χρήση auipc + jalr → διακλάδωση σε σχετική προς τον PC διεύθυνση

Παράδειγμα: x86-64 (64-bit mode)

- Εντολές διακλάδωσης χωρίς συνθήκη
 - (σχετική) $PC \leftarrow PC +$ ακέραια σταθερά των 8 bits (μετατόπιση -128...+127 bytes) ή των 32 bits (μετατόπιση $\pm 2GB$)
 - (απόλυτη) $PC \leftarrow$ 64 bits από έναν καταχωρητή ή διεύθυνση μνήμης
 - διακλαδώσεις ειδικού σκοπού
- Εντολές διακλάδωσης υπό συνθήκη
 - Η συνθήκη υπολογίζεται από προηγούμενη εντολή
 - Χρήση **flags** για τη μεταφορά του αποτελέσματος της σύγκρισης στην εντολή διακλάδωσης που ακολουθεί
 - Εάν ισχύει η ζητούμενη συνθήκη $\rightarrow$ προστίθεται στον PC μια ακέραια σταθερά (offset με πρόσημο) των 8 ή 32 bits

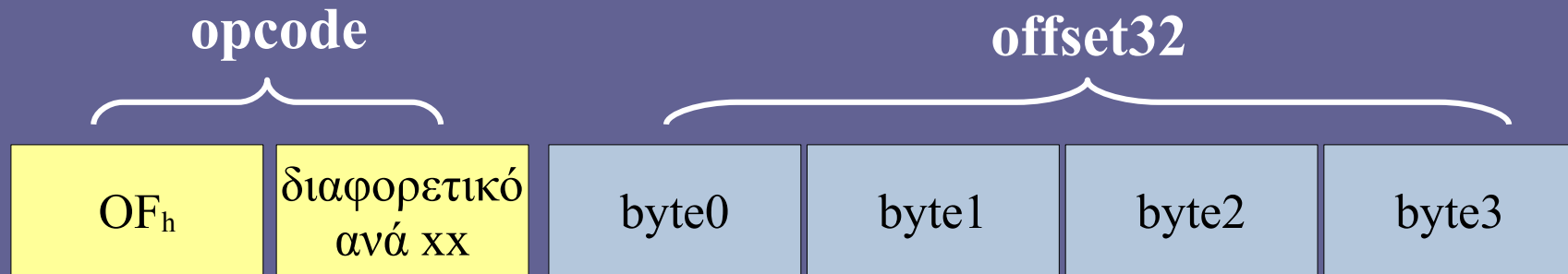
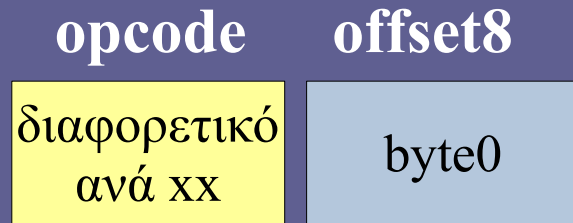
Σημαίες κατάστασης (flags)

- Σε αρχιτεκτονικές που εκτελούν διακλάδωση υπό συνθήκη με βάση το αποτέλεσμα **προηγούμενης εντολής**
- Τα flags είναι μια **ομάδα bits** που αναφέρουν την **κατάσταση** της μονάδας επεξεργασίας **μετά την εκτέλεση μιας εντολής**
 - Κάθε εντολή, ανάλογα με τη λειτουργικότητά της, επηρεάζει ορισμένα μόνο flags
- Τα πιο κοινά flags:
 - **(Z)**ero flag = μηδενικό αποτέλεσμα (της προηγούμενης πράξης)
 - **(S)**ign flag = αρνητικό αποτέλεσμα
 - **(C)**arry flag = ύπαρξη τελικού κρατουμένου
 - **o(V)**erflow flag = ένδειξη υπερχείλισης (αριθμοί με πρόσημο)
- Οι εντολές διακλάδωσης υπό συνθήκη εξετάζουν την τιμή ορισμένων flags για να αποφασίσουν αν θα γίνει διακλάδωση ή όχι

x86-64 Jxx instructions

- Έλεγχος αν κάποιο flag έχει τεθεί ή όχι
 - JC/JNC, JZ/JNZ, JS/JNS, JO/JNO
- Έλεγχος ισότητας δύο αριθμών
 - JE (z = 1)/JNE (z = 0)
- Αποτέλεσμα σύγκρισης δύο αριθμών με πρόσημο
 - JL (s $\neq$ v)/JGE (s = v), JG (z = 0 and s = v)/JLE (z = 0 or s $\neq$ v)
- Αποτέλεσμα σύγκρισης δύο αριθμών χωρίς πρόσημο
 - JB (c = 1)/JAE (c = 0), JA (c = 0 and z = 0)/JBE (c = 1 or z = 1)

x86-64 Jxx instruction formats (64-bit mode)



Εάν ισχύει η συνθήκη xx (έλεγχος flags), τότε στον PC προστίθεται το offset (αριθμός με πρόσημο, 8 ή 32 bits)

Γενική διαδικασία κλήσης συνάρτησης

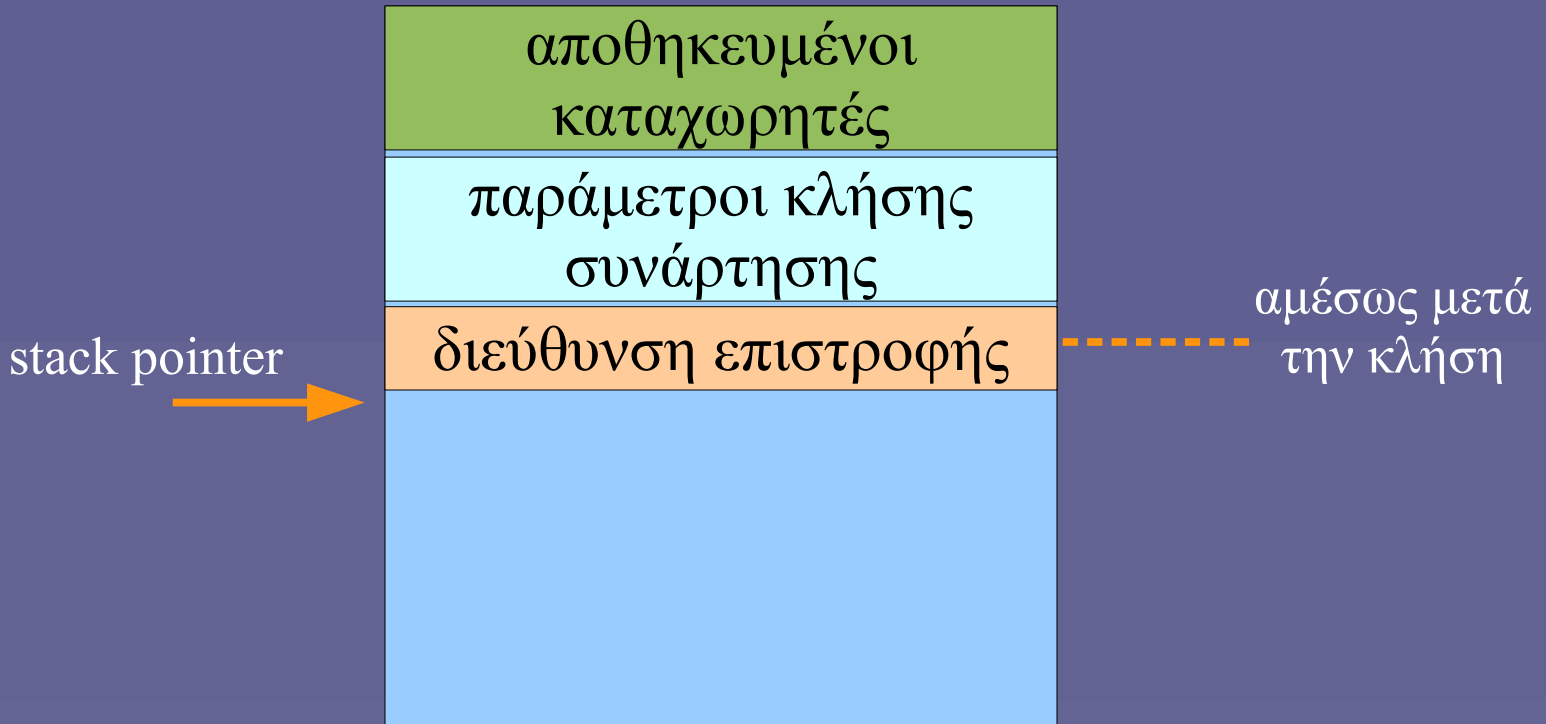
- Η καλούσα συνάρτηση
 - Αποθηκεύει καταχωρητές που χρησιμοποιεί και πιθανόν να αλλάξει η καλούμενη συνάρτηση
 - Περνάει τις παραμέτρους κλήσης σε καταχωρητές ή στο stack frame
 - Χρησιμοποιεί εντολή μηχανής τύπου “call”
 - Διακλάδωση στην καλούμενη συνάρτηση
 - Με ταυτόχρονη αποθήκευση της διεύθυνσης επιστροφής (stack frame ή καταχωρητές)
- Μετά την επιστροφή
 - Αποκατάσταση αποθηκευμένων τιμών καταχωρητών

Γενική διαδικασία κλήσης συνάρτησης (2)

- Η καλούμενη συνάρτηση
 - Δημιουργεί χώρο για τις τοπικές μεταβλητές στο stack
 - Υπολογίζει τη νέα τιμή του frame pointer (fp) και αποθηκεύει την παλιά
 - Αποθηκεύει καταχωρητές που πιθανόν να αλλάξει
- Στη συνέχεια εκτελείται ο κώδικας της συνάρτησης
- Πριν το τέλος
 - Αποδέσμευση χώρου τοπικών μεταβλητών, αποκατάσταση τιμών καταχωρητών και αποθηκευμένου fp
 - Χρήση εντολής μηχανής τύπου “return”
 - Διακλάδωση στην αποθηκευμένη διεύθυνση επιστροφής

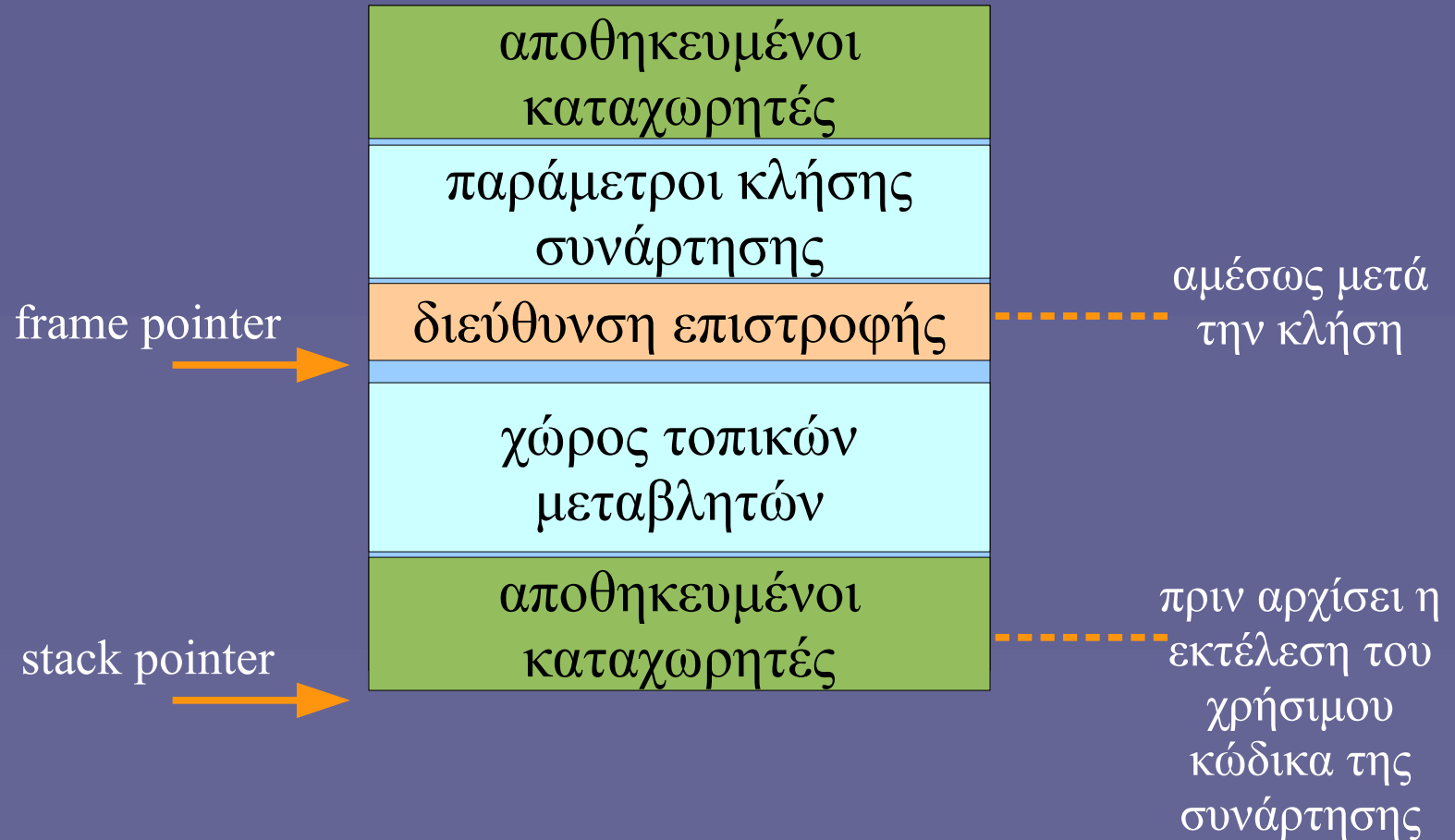
Παράδειγμα stack frame

(η συνάρτηση μόλις έχει κληθεί)



Παράδειγμα stack frame

(κατά την έναρξη εκτέλεσης του χρήσιμου κώδικα της συνάρτησης)



“Calling convention”

- Ο ακριβής τρόπος κλήσης των συναρτήσεων
 - Ποιες εντολές και ποιοι καταχωρητές χρησιμοποιούνται
 - Που αποθηκεύονται τα ορίσματα της κλήσης
 - Η ακολουθία ενεργειών πριν και μετά την κλήση/επιστροφή
 - Πώς είναι η ακριβής μορφή του stack frame
- Εξαρτάται από την αρχιτεκτονική (ISA) και το λειτουργικό σύστημα
 - Application Binary Interface (ABI)

Συναρτήσεις και RISC-V ISA

- Δεν υπάρχουν ειδικοί καταχωρητές για τη στοίβα (stack) που να επιβάλλονται από το υλικό
 - Οι εντολές δεν χρησιμοποιούν συγκεκριμένους καταχωρητές
- Κάποιοι καταχωρητές γενικού σκοπού χρησιμοποιούνται «κατά σύμβαση» στη διαδικασία κλήσης των συναρτήσεων
 - x1 → ra (return address)
 - x2 → sp (stack pointer)
 - x8 → fp (ή s0) (frame pointer)
 - x10-x17 → a0-a7 (argument registers, ορίσματα και επιστρεφόμενη τιμή)

Συναρτήσεις και RISC-V ISA

- «Κατά σύμβαση» η καλούμενη συνάρτηση πρέπει να διατηρήσει την τιμή ορισμένων καταχωρητών
 - Να τους αποθηκεύσει και να τους αποκαταστήσει στην αρχική τιμή πριν επιστρέψει
 - **sp**, **fp** και ορισμένοι άλλοι καταχωρητές
 - Ο κώδικας που εκτελεί τα παραπάνω ονομάζεται «**πρόλογος**» και αντίστοιχα «**επίλογος**» της συνάρτησης
 - Περικλείει τον κώδικα του χρήστη στη συνάρτηση

Διαχείριση στοίβας (runtime stack)

- Η στοίβα «μεγαλώνει» προς χαμηλότερες διευθύνσεις
 - π.χ. `addi sp, sp, -32` (δέσμευση χώρου για stack frame)
 - και `addi sp, sp, 32` (αποδέσμευση χώρου stack frame πριν την επιστροφή)
- Η προσπέλαση των τοπικών μεταβλητών στο stack frame γίνεται με τη βοήθεια του fp (ή αλλιώς, s0)
 - π.χ. `sw a1, -20(s0)`
 - Λειτουργία: `a1` $\rightarrow$ `mem[s0 - 20]`

Εντολές κλήσης και επιστροφής

- Κλήση συνάρτησης με εντολή **jal** (jump and link)
 - **jal ra, funcaddr** (ψευδοεντολή **call** funcaddr)
 - Λειτουργία: **ra** $\leftarrow$ pc + 4 (next instruction), pc $\leftarrow$ pc $\pm$ offset to **funcaddr**
- Επιστροφή με εντολή **jalr** (jump and link register)
 - **jalr x0, ra, 0** (ψευδοεντολή **jr ra** ή **ret**)
 - Λειτουργία: **x0** $\leftarrow$ pc + 4 (next instruction), pc $\leftarrow$ **ra** $\pm$ 0
 - Ο καταχωρητής **x0** έχει ειδικό ρόλο: όταν τον διαβάζουμε δίνει πάντα 0 και ό,τι γράφεται σε αυτόν χάνεται